COURIER CHARGE OPTIMIZATION

Case Study Report

This case study calculates courier charges using parcel weight and delivery distance. It also considers urgent delivery and restricted parcels, then compares normal and express charges.

1. Problem Statement

Develop a Python program to calculate courier delivery charges based on parcel weight and distance. The program should validate the entered values, check whether the parcel is restricted, calculate normal and express delivery charges, and recommend the service with the lower charge.

2. Objective

The objective is to apply Python input, variables, arithmetic operations, and conditional statements to calculate delivery costs and choose a suitable delivery option.

3. Assumptions

• Parcel weight is entered in kilograms and distance is entered in kilometres.

• Weight and distance must both be greater than zero.

• The normal delivery charge is calculated as weight × 20 + distance × 2.

• Express delivery adds ■100 for urgent delivery and ■50 for non-urgent delivery.

• Restricted parcels are not accepted by the service.

• The program selects normal delivery when its charge is less than or equal to the express charge; otherwise, it selects express delivery.

4. Algorithm

1. Start the program.

2. Read parcel weight, delivery distance, urgency, and restricted status.

3. If weight or distance is less than or equal to zero, display an invalid-input message.

4. If the parcel is restricted, display that service is unavailable.

5. Otherwise, calculate the normal charge using weight × 20 + distance × 2.

6. Calculate the express charge by adding ■100 for urgent delivery or ■50 for non-urgent delivery.

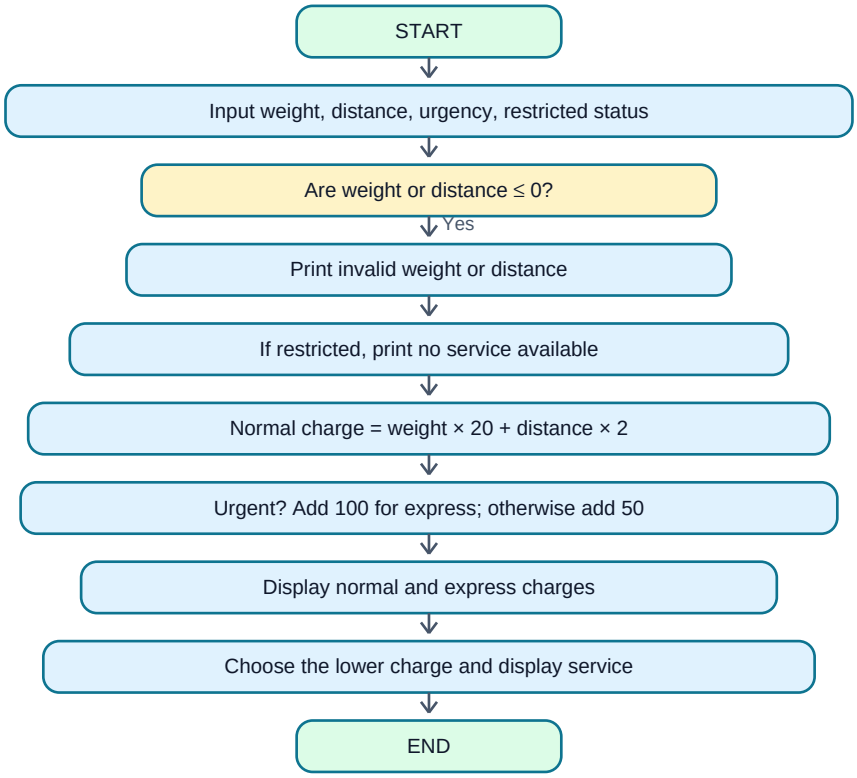
7. Display both normal and express delivery charges.

8. Compare the charges and select normal delivery if it is cheaper or equal; otherwise select express delivery.

9. Stop the program.

5. Flowchart

The flowchart outlines the main processing steps, from input validation to charge calculation and service selection.



6. Pseudocode

```
BEGIN
    INPUT weight, distance, urgent, restricted

    IF weight <= 0 OR distance <= 0 THEN
        PRINT "Invalid weight or distance"
    ELSE IF restricted = "yes" THEN
        PRINT "No service available for restricted parcel"
    ELSE
        normal_charge ← weight × 20 + distance × 2

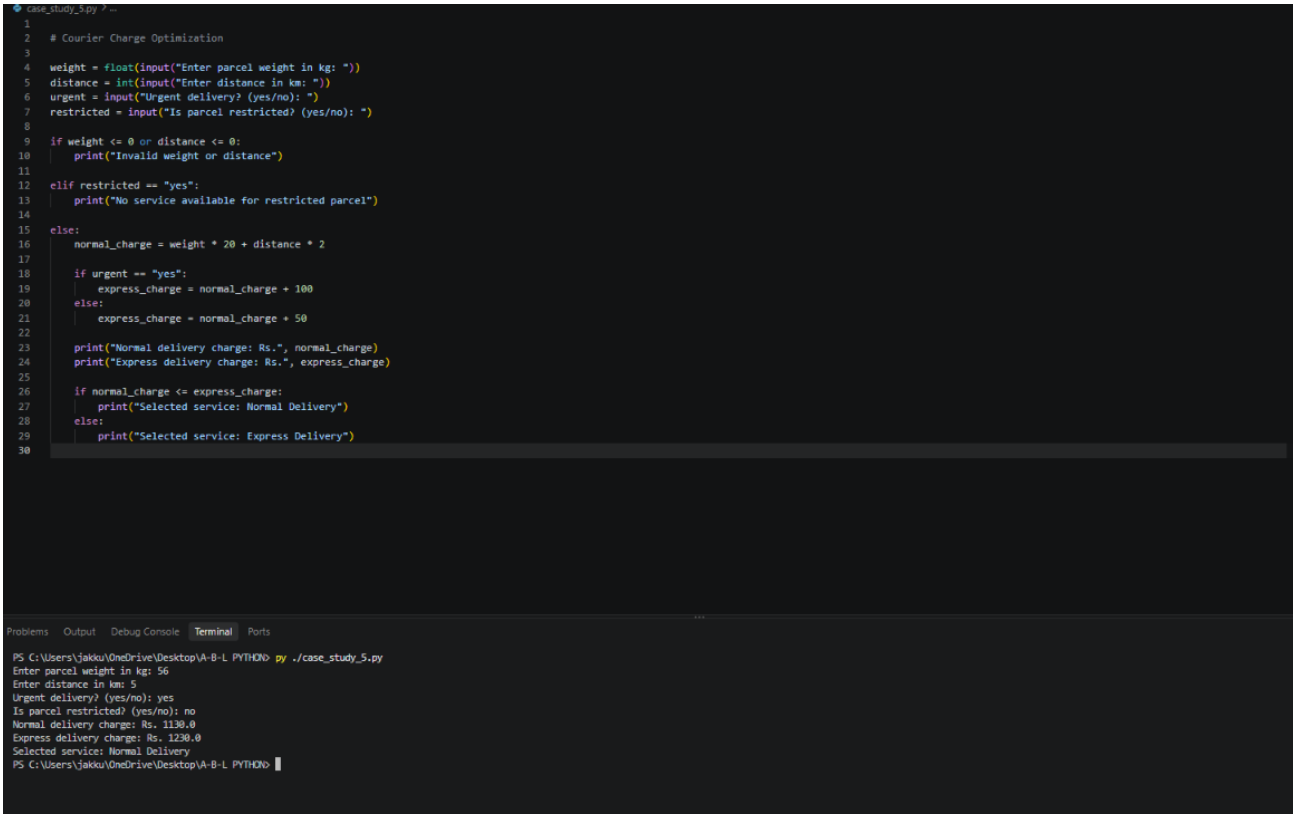
        IF urgent = "yes" THEN
            express_charge ← normal_charge + 100
        ELSE
            express_charge ← normal_charge + 50
        END IF

        PRINT normal_charge, express_charge

        IF normal_charge <= express_charge THEN
            PRINT "Selected service: Normal Delivery"
        ELSE
            PRINT "Selected service: Express Delivery"
        END IF
    END IF
END
```

7. Screenshot of Python Source Code and Output

The original screenshot of the Python code and terminal output is included below. For the shown inputs, the program calculates a normal charge of ■1,130 and an express charge of ■1,230, then selects Normal Delivery.



8. Conclusion

The Courier Charge Optimization program validates parcel details, calculates normal and express delivery costs, and chooses the lower-cost service. In the supplied example, the normal charge is ■1,130 and the express charge is ■1,230, so Normal Delivery is selected. The case study demonstrates how Python arithmetic and decision-making statements can support simple courier cost comparisons.